Appendix: Artifact Description

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

- C_1 We provide the first large-scale, trace-grounded characterization of AI network sensitivity across realistic workloads, fabrics, and scales up to 4,096 GPUs. We publicly release our trace dataset (˜2 TB of trace data on up to 4k GPUs), separate the latency, bandwidth and jitter contributions, and identify the regimes where each dominates the bottleneck.
- C_2 We develop and validate a compositional, linear-programming–based trace analysis methodology with signature-based caching that makes production-scale sensitivity studies feasible while preserving the dependency structure, regime boundaries, and sensitivity classifications of the underlying workload.
- C_3 We derive closed-form latency–bandwidth performance envelopes that expose safe relaxation regions and danger zones, and translate workload behavior into actionable provisioning guidance for AI-supercomputer fabrics.

B. Computational Artifacts

This paper is accompanied by two computational artifacts. Artifact A_1 contains the source code of the Composite-LP toolchain together with the packaged sensitivity-sweep outputs and the figure-generation scripts needed to regenerate every figure in the paper from the packaged data. Artifact A_2 is the companion collection of execution traces (GOAL format) used throughout the study.

- A_1 `https://github.com/tommasobo/SC_Tracing.git`
- A_2 `http://storage2.spcl.ethz.ch/traces/ai/`

Digital Object Identifiers for both artifacts will be minted at the AE stage, as allowed by the SC26 AD/AE policy.

Artifact ID	Contributions Supported	Related Paper Elements
A_1	C_1, C_2, C_3	Figs. 1, 3–10
A_2	C_1	Table II, Figs. 1, 3–10

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

Artifact A_1 ships every component needed to regenerate the paper's figures from raw GOAL traces, organized as the following subtrees:

- `solver/` — the per-collective LP solver and program-level composition code.
- `tools/LogGOPSim/` — the LogGOPSim source with a Makefile and a build-script wrapper.
- `pipeline/` — thin drivers (`run_composite_lp.py`, `run_lgs.py`, `demo.py`) that compose the solver and LGS into one pipeline.
- `data/` — the precomputed sensitivity CSVs, so figure reproduction is decoupled from solver-time requirements, plus a tiny demo GOAL trace for the pipeline demo.
- `scripts/` — one figure-generation script per paper figure; invoked by `reproduce_all.py`.

The toolchain ingests GOAL dependency graphs produced from NCCL traces (contribution C_1), decomposes the per-iteration workload into unique collective signatures, and builds a per-signature parametric linear program that it solves in a single dual-simplex sweep over latency or bandwidth; a lightweight program-level LP then composes the per-signature piecewise-affine slopes into an end-to-end sensitivity curve (contribution C_2). The sweep outputs are the envelopes and regime boundaries shown in Figures 1, 3–10 (contribution C_3). Figure 2 is a schematic of this pipeline and has no reproducible output.

Expected Results

The results should demonstrate three main claims of the paper. First, our Composite-LP methodology reproduces the end-to-end latency and bandwidth sensitivity of the Monolithic-LP baseline within approximately one percent on every workload for which Monolithic is feasible, while using orders of magnitude less memory and solve time. This memory advantage is what makes the sensitivity analysis feasible at production scales: the Monolithic LP would require tens of TB of RAM at 4,096 GPUs, whereas the Composite LP runs in roughly 200 MB.

Second, the derived latency-bandwidth envelopes should reveal qualitatively distinct regimes across workloads. Llama 70B training is expected to be bandwidth-dominated below $\sim$200 Gbps with a latency cliff only at tens of microseconds; Grok 314B at 4,096 GPUs is expected to be compute-bound across most of the studied range, with latency mattering only well above the regime of modern datacenter fabrics; vLLM Llama 8B inference is expected to be memory-bound with very little sensitivity to the network at all.

Third, the jitter analysis on Grok 314B is expected to show that the workload absorbs sparse jitter with sub-percent slowdown but degrades sharply once a large fraction of messages is perturbed, supporting the provisioning conclusions drawn in the paper.

All figures reported in the paper were produced by the same scripts included in A_1.

Expected Reproduction Time (in Minutes)

We distinguish three levels of reproduction.

Figure reproduction (from packaged CSVs; the default path for reviewers).

- Artifact setup: ˜2 min to clone the repository and install Python dependencies via `pip install -r requirements.txt`.
- Artifact execution: ˜1 min to run `python3 reproduce_all.py` on a standard laptop; each individual figure script completes in under 15 s.
- Artifact analysis: immediate. Figures are written to `figures/` as PDF and PNG; visual comparison with the paper is sufficient.

Pipeline demo (builds LogGOPSim from source and replays the shipped demo GOAL trace through the LGS stage).

- Additional setup: ˜30 s on first invocation to compile LogGOPSim via `pipeline/build_tools.sh`.
- Additional execution: ˜10 s to run `python3 reproduce_all.py --pipeline` end-to-end (the LGS replay of the demo GOAL is dominated by txt2bin conversion and runs in well under a second per latency point).

Data reproduction (from raw GOAL traces in A_2 to the packaged sensitivity CSVs in A_1). Running the end-to-end Composite-LP pipeline from traces to CSVs requires a workstation with a licensed Gurobi installation. We report approximate end-to-end wall times below (on an AMD EPYC host with 128 cores and 380 GiB of RAM):

- Trace download from A_2: dominated by the user's bandwidth (˜2 TB total across all workloads; individual workloads are 1–10 GiB for Llama / vLLM and up to ˜1.3 TiB for Grok 314B at 4,096 GPUs).
- GOAL parsing and signature extraction: ˜5 min for Llama 70B, ˜2 min for vLLM, ˜30 min for Grok 314B.
- Composite-LP sweep over latency or bandwidth: ˜1–5 min per workload; Grok 4,096 GPUs finishes in under 10 min on a single socket.
- Monolithic-LP cross-check (only feasible for small scales): ˜83 min at Llama 8B / 16 GPUs; infeasible beyond ˜1,024 GPUs due to memory (see Figure 7 of the paper).

Reviewers who opt for the default figure-reproduction path do not need to execute any part of the data-reproduction stage.

Artifact Setup (incl. Inputs)

Hardware: Figure reproduction requires no specialized hardware: any x86 or ARM machine with ∼1 GiB of free RAM is sufficient. The upstream composition and solver stages used to produce the packaged CSVs used an AMD EPYC host (128 cores, 380 GiB RAM) with Gurobi; scale-out tracing of Grok 314B (up to 4,096 GPUs) was performed on the Microsoft Azure ND GB200 v6 cluster, and Llama / vLLM tracing was performed on the CSCS Alps supercomputer (NVIDIA GH200, Slingshot-11). None of that infrastructure is needed to reproduce the figures from the packaged data.

Software: Figure reproduction requires Python 3.8 or later and three widely-available Python packages:

- `matplotlib` >= 3.5
- `numpy` >= 1.21
- `pandas` >= 1.3

A `requirements.txt` is shipped with the artifact.

The optional `--pipeline` path additionally builds LogGOPSim from its bundled source, which requires `g++`, `gengetopt`, and `re2c` (Ubuntu: `apt-get install g++ gengetopt re2c`).

Regenerating the Composite-LP or Monolithic-LP sensitivity CSVs from raw GOAL traces additionally requires a Gurobi installation (version 10.0 or later). Gurobi is commercial software, but the free academic license offered at gurobi.com/academia is sufficient to reproduce every LP solve reported in the paper. The Gurobi dependency applies *only* to the LP stage of data reproduction; the figure-reproduction path and the LGS pipeline demo do not invoke any solver.

Datasets / Inputs: All inputs required to regenerate the figures are shipped inside A_1 under `data/`: ∼6 MiB of CSV files containing the latency and bandwidth sensitivity sweeps, memory footprints, and hardware reference measurements for the three workloads. The full collection of raw execution traces is published separately as A_2 (˜2 TB, GOAL format). A_2 is not required for figure reproduction; it is the input to the LP solver stage that generated the packaged CSVs in A_1.

Installation and Deployment: Clone the repository (the artifact lives on the `main` branch, which is the default), create a Python virtual environment, and install the three runtime dependencies:

```
git clone -b main \
  https://github.com/tommasobo/SC_Tracing.git
cd SC_Tracing
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

No compilation is required. The figure-reproduction path uses only Python and the packaged CSVs under `data/`.

Artifact Execution

The end-to-end path from cloning the repository to producing every paper figure is three commands:

```
git clone -b main \
  https://github.com/tommasobo/SC_Tracing.git
cd SC_Tracing
pip install -r requirements.txt
python3 reproduce_all.py
```

Concretely, the workflow has three stages:

- T_1 — Install Python dependencies with `pip install -r requirements.txt`.
- T_2 — Run `python3 reproduce_all.py`, which iterates over the paper's figure list and invokes one Python script per figure. Each script reads CSV files from `data/` and writes the corresponding PDF and PNG to `figures/`.

- T_3 (optional) — Run `reproduce_all.py --pipeline`, which first builds LogGOPSim from source under `tools/LogGOPSim/` and replays the shipped demo GOAL trace through the LGS stage, then proceeds with T_2. This exercises the same LGS machinery used at paper scale. Expected wall time: $<$1 min on top of the figure-reproduction path.

Task dependencies: $T_1 \rightarrow T_2$ and $T_1 \rightarrow T_3 \rightarrow T_2$. No parameter sweeps or randomness are involved in T_2: every figure is deterministic given the packaged CSVs, so repetition is not required. Users may reproduce a subset of figures by passing their paper numbers via `--only`, e.g. `python3 reproduce_all.py --only 3 4 5`.

Driving the pipeline on production-scale traces. To run the Composite-LP sweep or LGS replay on the actual paper workloads, download one of the per-workload trees from A_2 (each ships a GOAL file plus a `comm_dep.csv` that disambiguates sends/recvs) and invoke:

```
python3 pipeline/run_composite_lp.py \
  --goal <workload>/output.goal \
  --comm-dep <workload>/comm_dep.csv \
  --out out/<workload>_composed.csv \
  --l-min 0 --l-max 1000000 --step 50000
python3 pipeline/run_lgs.py \
  --goal <workload>/output.goal --L 1000
```

The Composite-LP stage requires a Gurobi 10.0+ installation; the LGS stage does not.

Precomputed outputs for resource-intensive stages. Several upstream stages of the pipeline are too expensive for a typical reviewer machine — in particular, LogGOPSim replay of the Grok 4,096-GPU trace, and the Monolithic-LP solves used as baselines in Figures 5 and 7 (the largest of which requires several hundred GiB of RAM). To make figure reproduction independent of access to such machines, we ship the *outputs* of these runs as precomputed CSVs inside A_1: the LP sensitivity sweeps, the LGS points, the hardware reference measurements, and the Grok memory footprints all live under `data/` and are consumed directly by the figure scripts. Reviewers can therefore validate all paper figures from the packaged artifact without re-executing any LP solve or packet-level simulation.

Artifact Analysis (incl. Outputs)

Running `reproduce_all.py` writes the following files to `figures/`:

- `fig_2d_sensitivity_workloads.pdf` — paper Figure 1.
- `fig3_sensitivity_1x4.pdf` — paper Figure 3.
- `fig_mixed_16n_ch1.pdf` — paper Figure 4.
- `fig5_llama7b.pdf` — paper Figure 5.
- `fig_3x3_sensitivity.pdf` — paper Figure 6.
- `fig6_grok_memory.pdf` — paper Figure 7.
- `fig_network_perf_combined.pdf` — paper Figures 8 and 9 (combined).
- `fig_jitter_3panel.pdf` — paper Figure 10.

Each generated PDF should be visually identical to the corresponding figure in the paper. A reviewer may perform the comparison by opening each PDF next to the paper manuscript; the script `reproduce_all.py --list` prints the figure-to-script mapping for easy cross-referencing.

B. Computational Artifact A_2

Relation To Contributions

Artifact A_2 is the complete collection of NCCL execution traces used throughout the paper, converted to the GOAL format, and covering Llama 3.3 training (16–128 GPUs), Grok 3 training (16–4,096 GPUs), and Llama 3.1-8B inference under vLLM (8 GPUs). By publishing this dataset we fulfill contribution C_1: enabling open, reproducible trace-driven network-sensitivity research.

Expected Results

Each GOAL file in A_2 can be replayed with LogGOPSim or any other GOAL-compatible simulator, and is the direct input to the Composite-LP toolchain in A_1. The expected output of replaying a trace is the corresponding $T(L)$ or $T(G)$ curve that appears in the packaged CSVs of A_1. Table II of the paper reports reference latency and bandwidth values drawn from these traces.

Expected Reproduction Time (in Minutes)

No reproduction time applies: A_2 is a data release. Download time dominates and depends on the user's bandwidth (2 TB total; individual traces range from a few MiB for vLLM inference to $\sim$1.3 TiB for Grok 314B at 4,096 GPUs).

Artifact Setup (incl. Inputs)

Hardware: None. A_2 is a data archive. Replaying its GOAL files end-to-end under a packet-level simulator requires an x86 host with enough RAM to hold the target trace (up to $\sim$200 GiB for the largest Grok run); replaying under LogGOPSim or our Composite-LP toolchain requires only a few GiB.

Software: A GOAL-compatible trace reader. All scripts in A_1 that need to re-process traces use the reader shipped with the LogGOPSim sources referenced in the main paper.

Datasets / Inputs: Self-contained data release.

Installation and Deployment: Only a subset of the traces published under `http://storage2.spcl.ethz.ch/traces/ai/` is consumed by the figures of this paper: Llama 3.3 training at 16–128 GPUs, Grok 3 training at 16–4,096 GPUs, and Llama 3.1-8B inference under vLLM at 8 GPUs (listed in Table II of the paper). We therefore recommend fetching only those specific per-workload subtrees rather than mirroring the full listing. Example for a single workload:

```
wget -r -np -nH --cut-dirs=2 \
  http://storage2.spcl.ethz.ch/traces/ai/grok3/
```

No build step is required.

Artifact Execution

A_2 is a static data release; there is no execution step. The intended downstream workflow is `(trace` $\in A_2$`)` $\rightarrow$ `LP solver (shipped in` A_1`)` $\rightarrow$ `sensitivity CSV (also shipped in` A_1`)` $\rightarrow$ `figure`. Only the last two stages are exercised by the reproduction workflow in A_1.

Artifact Analysis (incl. Outputs)

The artifact itself is the output. File integrity can be verified against the SHA-256 manifest shipped alongside the trace archive.